

Documento di implementazione

1. Descrizione e motivazione dell'uso di Singleton

Il Singleton è un design pattern creazionale che garantisce che una classe abbia una sola istanza durante l'intero ciclo di vita dell'applicazione e fornisce un punto di accesso globale a tale istanza. L'utilità di tale pattern sta nell' evitare la presenza di più istanze che sarebbe dannosa o inutile, così da centralizzare la gestione di una risorsa condivisa, specie se tale risorsa è critica.

Gli elementi fondamentali dell'implementazione del Singleton sono :

Costruttore privato : impedisce la creazione dell'oggetto dall'esterno tramite new.

Campo statico privato : Contiene l'unica istanza della classe.

Metodo getInstance : restituisce l'istanza, creandola solo se non esiste già.

L'uso di Singleton nell'applicazione SimpleInvest è presente in DBManager.java ed è applicato all'omonima classe, che ha il compito di gestire la connessione al database SQLite.

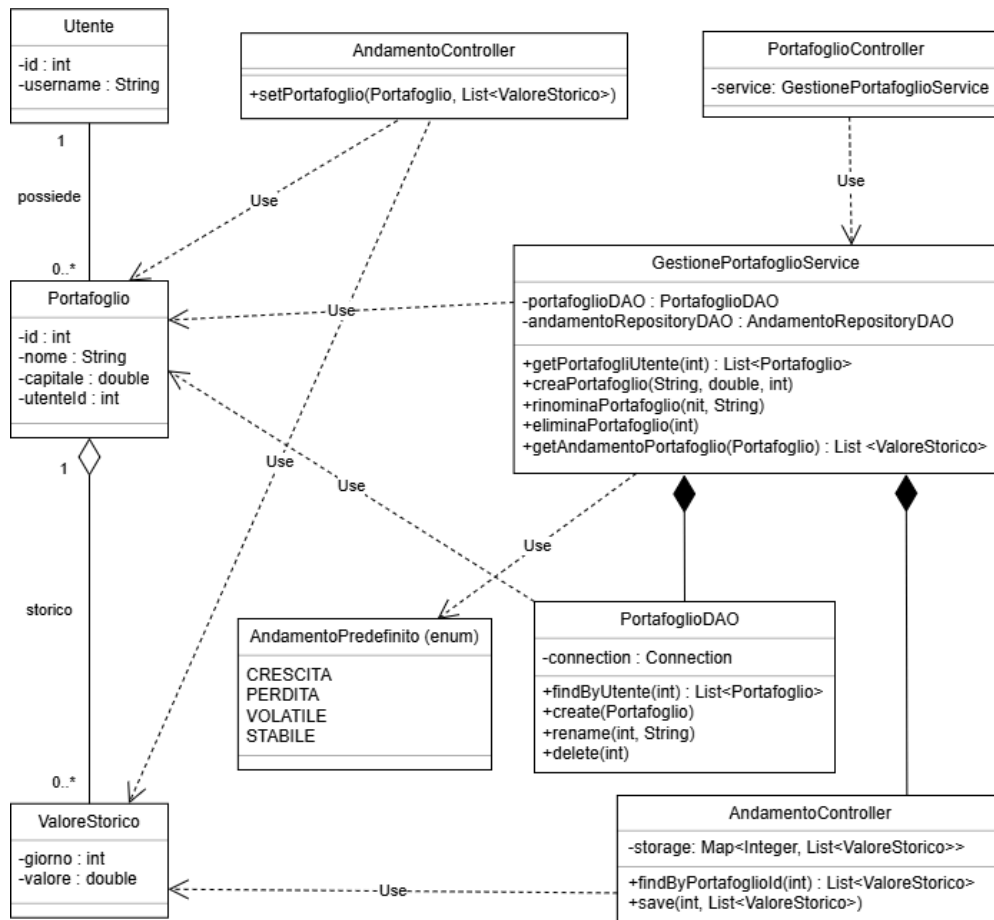
La connessione al database è una risorsa costosa e la creazione di più connessioni porterebbe a spreco di risorse, problemi di sincronizzazione e incoerenza nell'accesso ai dati. Il Singleton assicura unicità della connessione, centralizzazione della logica di accesso al database e maggiore controllo del codice.

Estratti dal codice :

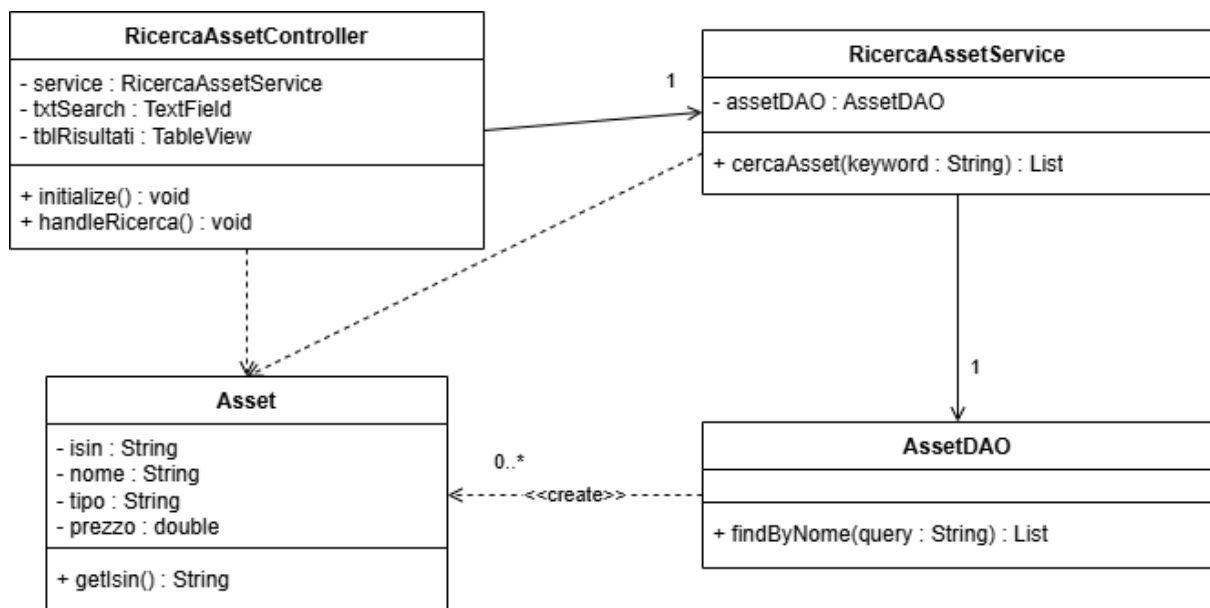
- `private static DBManager instance;` → istanza unica della classe
- `private DBManager() {` → costruttore privato
 `// inizializzazione della connessione`
 `}`
- `public static DBManager getInstance() {` → metodo getInstance()
 `if (instance == null) {`
 `instance = new DBManager();`
 `}`
 `return instance;`
 `}`

2. Diagrammi delle classi software

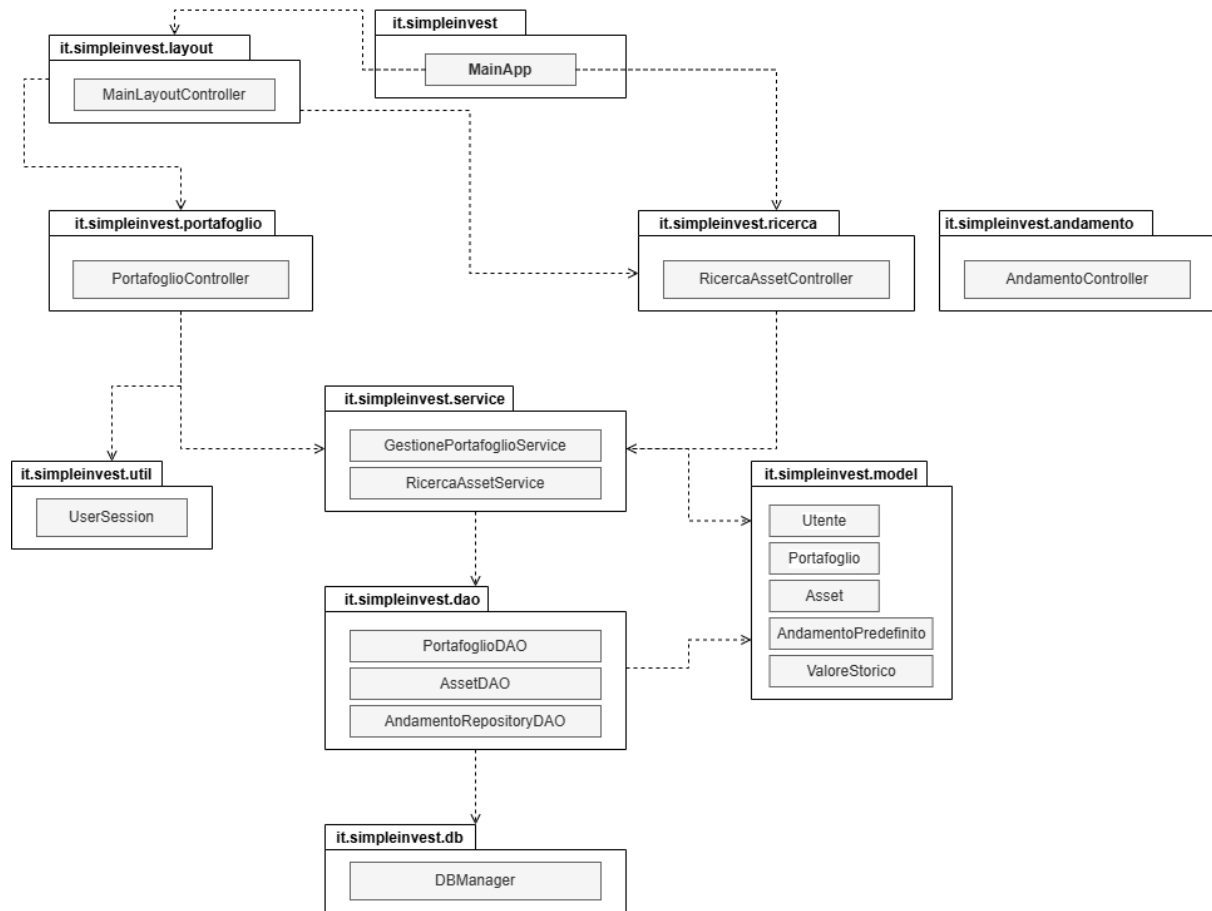
2.1 Diagramma classi software per UC3 (Donati) :



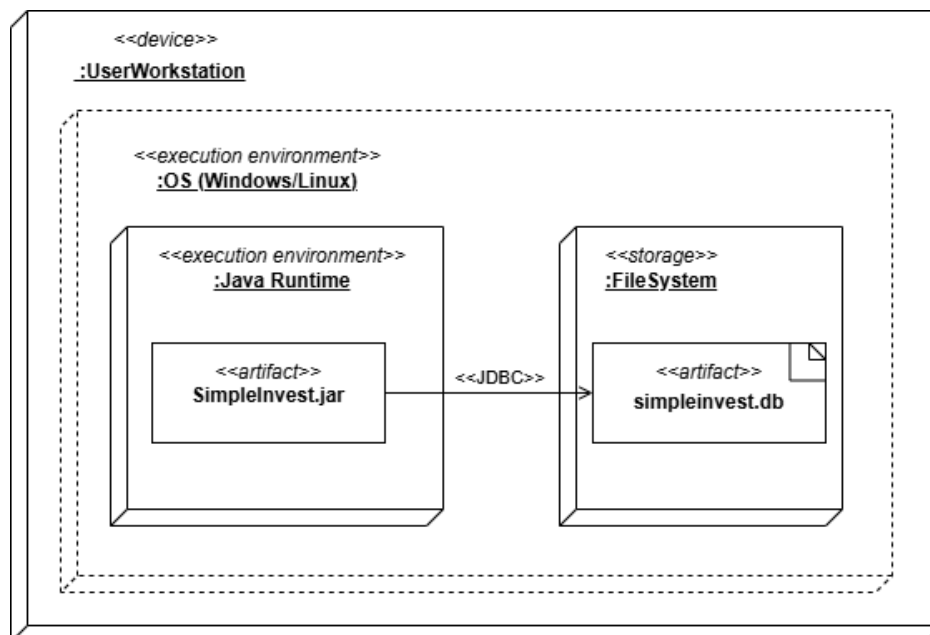
2.2 Diagramma classi software per UC5 (Levati) :



3. Diagramma dei package

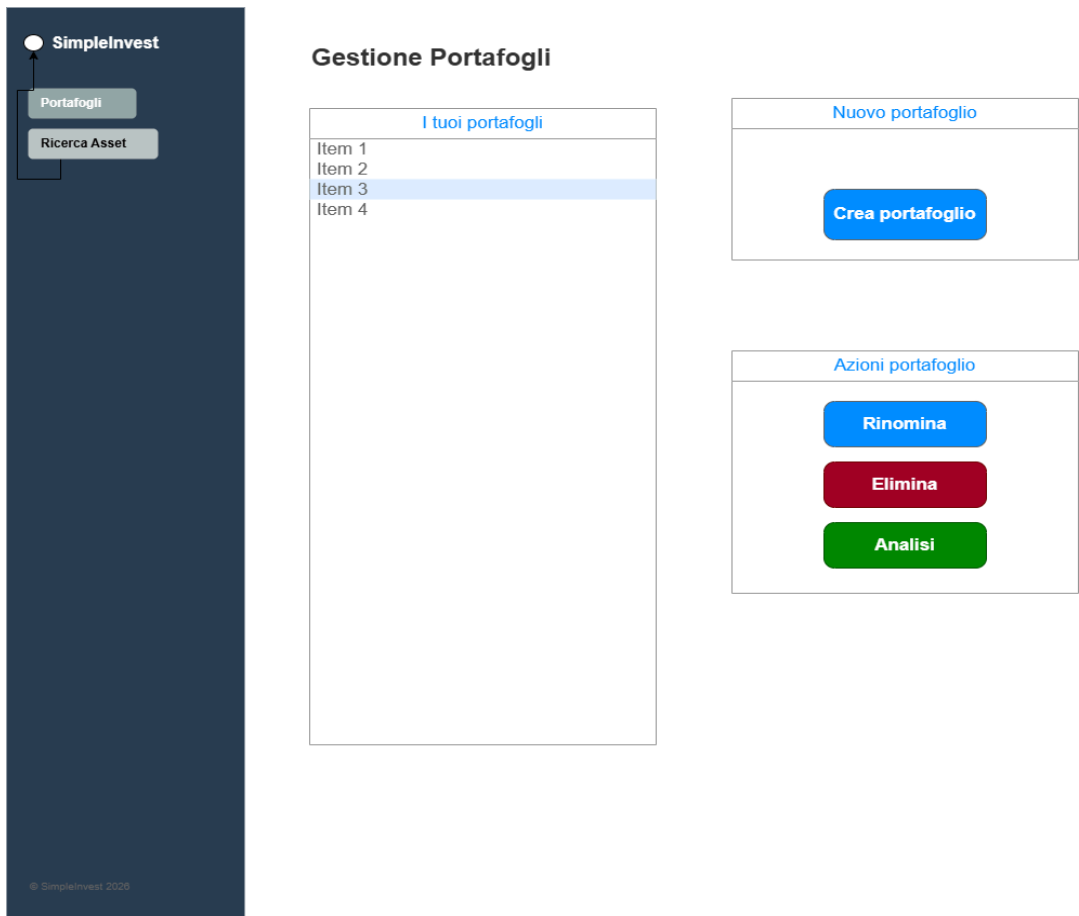


4. Diagramma di deployment



5. Schermate di mockup

5.1 Schermata di mockup per UC3 (Donati) :



5.2 Schermata di mockup per UC5 (Levati) :

